AUTOMATION PORTFOLIO — TECHNICAL RECORD

SOPHON FINANCE SYSTEMS

AI-Driven Finance & Accounting Automation | github.com/sophonfinance-wq/finance-automation-portfolio

MONTH-END CLOSE AUTOMATION

A deterministic, seeded Python engine that runs a full multi-entity month-end close on fictional data — computing the recurring journal entries, enforcing tie-out controls, and proving nothing slipped past via a ten-control validation layer.

Report No.	SFS-ENG-001	**Package**	close_engine
System	Month-End Close Automation	**Data**	Fictional / seeded
Run	`python -m close_engine --period 2026-03 --out ./output`	**Status**	Public — portfolio record

1.0 OVERVIEW AND SCENARIO

Running the seeded synthetic group (period 2026-03, seed 2026: entities DH/Demo Holdings LLC, MF/Maple Fund LP, BW/Birchwood Op Co), the engine posts seven recurring entries that tie out to a balanced trial balance of Dr 3,587,237.50 = Cr 3,587,237.50. The value is the failure modes it catches: the bundled guardrail demo injects twelve classic close errors one at a time and shows each caught by the specific control designed for it — for example a fully depreciated asset that keeps depreciating (caught by C4 asset-life guard), a one-sided intercompany note whose lender mirror was dropped (C2), an allocation driven by a year-to-date balance instead of the month's activity (C5), and a signed-off prior period quietly rewritten (C10). Each of these looks like a clean close until a control re-derives the number independently and flags the discrepancy.

2.0 WHAT THE SYSTEM AUTOMATES

- Prepaid amortization on a straight-line basis over each item's service period, relieving the prepaid asset (acct 1400)
- Fixed-asset depreciation (straight-line, monthly, no salvage), with the charge stopping at end of useful life
- Deferred-rent straight-lining plus CAM for a shared lease, with non-holder entity shares routed through intercompany due-to (2800) / due-from (1800) so each entity leg self-balances
- Management-fee accrual that nets any in-month cash payment against the monthly fee
- Related-party note interest accrual (simple monthly interest) posted with the lender/borrower intercompany mirror
- G&A cost allocation by a fixed basis-point ratio validated to sum to exactly 100% (10000 bps), using largest-remainder so no penny is lost
- Insurance premium allocation across shared policies with a mid-year renewal step-up that switches to the renewal rate in the renewal month
- Assembly of the JE register, updated trial balance, close report, and machine-readable control-findings JSON; an autonomous loop can additionally resync a drifted register back to the seeded system of record

3.0 OPERATING PROCEDURE

1. Generate a fully seeded, fictional dataset for the period: chart of accounts, three operating entities, an opening trial balance that balances per entity, and the source sub-ledgers (prepaids, fixed assets, leases, notes, management fees, G&A pool, insurance policies)
2. Load the opening trial balance into an integer-cent general ledger keyed by (entity, account)
3. Refuse up front any allocation split map that fails validation (must sum to 10000 bps and name only in-group entities) so the close never crashes mid-run
4. Compute each of the seven recurring entries together with a formula-driven backing schedule
5. Enforce the balance control on every entry — debits == credits in aggregate and within each entity leg — recording any out-of-tie entry as 'refused' instead of posting it
6. Tie each declaring schedule back to its GL account balance (e.g. prepaid schedule to acct 1400, insurance to acct 1450)
7. Run the Close Sentinel controls C1–C10, each re-deriving its expectation from the raw sub-ledgers independently of the posting code, and grade findings CRITICAL / WARN / INFO
8. Write the JE register (.md/.json), trial balance, close report with a Control findings section, and a sentinel findings JSON
9. Exit non-zero if the close is not clean (any refused entry, failed schedule tie, or unbalanced TB) or if any control raises a CRITICAL finding, so it can gate a pipeline

4.0 CONTROLS AND SAFEGUARDS

- Determinism and reproducibility: stdlib-only, seeded random streams so a given seed produces byte-identical inputs and outputs; all money held as integer cents so tie-outs are exact
- Read-only, independent verification: each of the ten controls is a pure function that never mutates the close and re-derives its expectation from the raw sub-ledgers, not the engine's intermediate math (e.g. C1 re-sums lines itself; C4 uses the posted register and sub-ledger in-service date, never the workpaper schedule)
- Independent shadow recomputation (C9): a minimal second implementation recomputes every expected amount from raw data and the posted register must agree to the cent before the close counts
- Hard posting gate: the engine refuses to post an out-of-tie entry (aggregate or per-entity leg) and records it as refused; any CRITICAL Sentinel finding makes the exit code non-zero
- Allocation integrity: ratios are validated to sum to exactly 100% (10000 bps) before posting, and largest-remainder splits guarantee the per-entity detail crossfoots exactly to the monthly total (C6, C8)
- Period lock (C10): each closed register is sealed with a SHA-256 hash of its canonical JSON; a later deterministic recompute detects any post-sign-off mutation
- Separation of duties in the autonomous loop: findings the loop has no authority to act on — a tampered locked prior period (quarantine) and a broken opening carryforward (halt) — are held and logged rather than auto-overwritten

5.0 VERIFICATION AND TESTING

Correctness is proven, not asserted. The repository states a passing pytest suite of 5,542 tests (of which 3,742 are Sentinel tests) covering JE balancing in aggregate and per entity, straight-line math, allocations summing to 100% with no penny lost, insurance re-rating at the renewal step-up, out-of-tie refusal, period roll-forward, and full seed determinism. The `--demo-guardrails` command runs a clean baseline that must

produce zero findings, then injects twelve seeded faults and asserts each is caught by its mapped control at a qualifying severity, exiting 0 only if the baseline is clean and all twelve are caught. The CLI's non-zero exit on any unclean close or CRITICAL finding, and the autonomous loop's verdict-as-exit-code, make correctness checkable in CI.

6.0 KEY FACTS

Attribute	Value
Recurring-entry classes automated	7
Close Sentinel deterministic controls	10 (C1–C10)
Fault injectors in the guardrail demo	12 (covering all 10 controls)
Operating entities in the seeded group	3 (DH, MF, BW)
Test suite (README-stated)	5,542 passing (3,742 Sentinel)
Balanced trial balance (2026-03, seed 2026)	Dr 3,587,237.50 = Cr 3,587,237.50

This record documents a self-contained system in the Sophon Finance Systems automation portfolio. All data is fictional and seeded; no employer or client workpaper, entity, methodology, path, or figure is reproduced. Prepared 14 July 2026.